

verify_leads.py – Complete Explanation

What is this file?

The uploaded file is compiled Python bytecode named `verify_leads.cpython-314.pyc`. It corresponds to a Python source file called `verify_leads.py`. The explanation below describes the program's functions, verification logic, scoring, input/output handling, and overall workflow based on the inspected compiled file.

Step-by-Step Explanation

- 1. Program purpose** – The program is a standalone lead-verification script. It takes a CSV containing leads, verifies them using available information and external sources, assigns a verification score and status, and writes the results to another CSV file.
- 2. OpenStreetMap / Nominatim** – The program uses the OpenStreetMap Nominatim search service to search for a lead using its agency name and location.
- 3. NOMINATIM_URL** – The program defines the Nominatim search endpoint as `"https://nominatim.openstreetmap.org/search"`.
- 4. USER_AGENT** – The program defines a User-Agent string, `"SkillUpLeadVerifier/1.0 (your-email@example.com)"`, to identify the application when making requests.
- 5. NOMINATIM_DELAY** – The program uses a delay of approximately 1.1 seconds between Nominatim requests so that it does not send requests too rapidly.
- 6. Verification keywords** – The program contains keywords related to detectives, investigations, security, lawyers, and police. These keywords help determine what type of organization a lead appears to be.
- 7. normalize_text(value)** – This function cleans text before comparison or searching. It normalizes the value, removes unnecessary whitespace, and converts text to a consistent form so comparisons are more reliable.
- 8. detect_profession(name, website, page_text)** – This function attempts to classify a lead based on its name, website, and webpage text. The possible categories include Police, Lawyer, Private Detective, Security, and Unknown.
- 9. fetch_website_text(url)** – This function attempts to access a lead's website using an HTTP request. It extracts readable webpage text by removing HTML elements such as scripts, styles, and tags. Errors are handled so that a failed website request does not crash the whole verification process.
- 10. search_nominatim(name, location)** – This function searches Nominatim using the lead's name and location and processes the returned JSON results. It can provide information about matching places.
- 11. check_website_for_keywords(url, name)** – This function checks a website, extracts its text, searches for relevant keywords, and uses that information to help determine the lead's profession and verification evidence.
- 12. name_similarity(name_a, name_b)** – This function compares two names after normalizing their words. It calculates a similarity value based on overlapping tokens, helping determine whether a lead name and a place name refer to the same organization.
- 13. score_lead(row)** – This is the main lead-verification function. It examines a lead and combines evidence from OpenStreetMap, name similarity, website information, keywords, phone presence, and detected profession to produce verification information and a score.
- 14. Nominatim name matching** – The program compares the lead name with an OpenStreetMap result. The inspected code contains a similarity threshold of 0.35 for this name-match evidence.
- 15. Nominatim keyword matching** – The program also checks relevant keywords in OpenStreetMap information. Verification sources can include indicators such as `nominatim_found`, `nominatim_name_match`, and `nominatim_keyword_match`.
- 16. Website verification** – If a lead has a website, the program attempts to access it and checks the HTTP response. Website evidence can include `website_exists` and `website_keyword`.

- 17. Phone verification** – The program checks whether a usable phone value is present and removes non-digit characters when examining it. The presence of a phone number can contribute evidence through `phone_present`.
- 18. Profession and scoring** – The detected profession is used as part of the verification information. The program handles categories including Police, Lawyer, Private Detective, Security, and Unknown.
- 19. Verification status** – The program defines three status ranges: `score >= 70` gives Verified; `40 <= score < 70` gives Needs Review; `score < 40` gives Unverified.
- 20. `main(argv)`** – The main function controls the complete verification process and uses command-line arguments to specify input, output, and optional sampling.
- 21. Input CSV** – The default input file is `scraped_leads.csv`. A different input file can be supplied using the `--input` command-line option.
- 22. Output CSV** – The default output file is `verified_leads.csv`. A different output file can be supplied using the `--output` command-line option.
- 23. `--sample`** – The program supports a `--sample` option that allows only a specified number of rows to be processed, which is useful for testing.
- 24. Pandas read_csv** – The program uses Pandas to read the input CSV into a DataFrame and then processes the leads from that DataFrame.
- 25. Process each lead** – The program processes leads individually, sending each row through the lead-scoring and verification logic.
- 26. Verification fields** – For each lead, the program creates information including `verification_status`, `verification_score`, `verified_sources`, `verified_at`, and `profession`.
- 27. Verification timestamp** – The program records when verification was performed using a timestamp based on UTC time.
- 28. Output DataFrame** – After processing the leads, the program creates the final DataFrame containing the original lead information together with the generated verification information.
- 29. Write final CSV** – The program uses Pandas to write the final verification results to the output CSV, which by default is `verified_leads.csv`.
- 30. Output information** – The resulting CSV contains the original lead information plus `verification_status`, `verification_score`, `verified_sources`, `verified_at`, and `profession`.

Overall Workflow

Scraped Leads CSV → Read using Pandas → Take one lead → Search OpenStreetMap → Compare names → Check website → Extract website text → Search keywords → Detect profession → Check phone → Combine verification signals → Calculate score → Assign status → Add timestamp → Process next lead → Save `verified_leads.csv`.

In Simple Words

This program is a lead-verification system. It takes scraped leads and checks them against available evidence such as OpenStreetMap results, website information, keywords, name similarity, and phone presence. It then calculates a verification score, assigns a status such as Verified, Needs Review, or Unverified, records the detected profession and verification time, and saves the results into a new CSV file.

The uploaded `.pyc` file is compiled bytecode, not the original human-readable source. The corresponding source file is expected to be `verify_leads.py`. When that source file is provided, the explanation can be made directly line-by-line from the actual source code.